PrivAI 2026 Course Projects: Rules and Project Ideas

Krishna Pillutla (IIT Madras)

Project Proposal Due: Friday, March 27, 2026 at 11:59 PM
Project Mid-Review Due: Friday, April 17, 2026 at 11:59 PM
Project Presentation: April 29 - May 4 (TBA)
Project Final Report Due: Sunday, May 11 at 11:59 PM

Instructions

- The course project can be performed in teams of 3 (preferred) or 2 students. The expected output is roughly proportional to the number of people in the team. On a case-by-case basis, we may allow individuals to work alone on their course project if it is based on their ongoing research project.
- Note that no extensions or late days are allowed for project deadlines.
- Updates to the documents are shown in blue. Last updated: March 17, 2026.

0.1 Types of Projects

The course project can be one of the three types (corresponding to Sections 1-3 below):

- An original research project: can be theory/applied/both. You are welcome to work on your research project, as long as it involves a component related to privacy in AI.
  The goal of this type of project is to write a report documenting independent research that would get accepted in the workshops accompanying top ML/AI conferences such as NeurIPS/ICML/ICLR. With additional effort past the end of the course, such papers can get accepted for publication at top ML/AI venues. (For example, the instructor's first scientific publication came out of a course project.)
- Implementation: benchmarking existing algorithms and open-sourcing by creating your package or contributing to existing ones.
- In-depth paper analysis: read and analyse the results of a theoretical paper, and reproduce the proofs in your own words.

0.2 Project Deliverables and Details

Project Proposal The project proposal should answer the following questions: (1) What exactly do you want to do? (2) Why is this problem important? In addition, the proposal should also identify relevant papers for the literature survey.
Note that coming up with a good list of relevant papers requires quite a bit of initial reading already. You do not write about everything you have read in this proposal report, but you have to list the relevant reading.

Project Mid-Report The mid-report should contain the following:

- Details of your literature survey: Papers and ideas must be explained mathematically, as we did in the lectures. The text must be written entirely on your own (LLMs are not allowed for writing, but you may use them for help with figures).
- The desired outcome: for the theoretical project, what is the final result you wish to prove? For an empirical project, what exact algorithms do you expect to implement, on what datasets, using what compute?
- Evidence of preliminary progress towards these goals: For theoretical projects, proofs of some intermediate lemmas, or a proof sketch breaking down the overall path to the full proof, to demonstrate progress towards these goals. For empirical projects, empirical results on some baselines and some simple toy datasets show preliminary progress.

Project Final Report You need to introduce the reader to your problem, and convince them about its importance. The report should also contain a detailed account of all your main results. Theoretical projects should give full and rigorous proofs (for research projects: you may use established results from existing textbooks and papers without proving them again; for paper-reading projects: proving previous results is necessary). Empirical projects should show detailed plots of numerical performance and should document the implementation and hyperparameter choices carefully.

Formatting Insturctions and Expected Length The project proposal, mid-review, and final report are each to use the NeurIPS LaTeX template, available at `https://media.neurips.cc/Conferences/NeurIPS2025/Styles.zip`. The length of each document (excluding references and supplementary materials) is:

- Project Proposal: 2 pages
- Project Mid-Review: 4 pages
- Final Report: 9 pages

Additional details such as details about the experimental setup or proofs can be provided as appendices to each of the reports above (with no restrictions on the page limit). Note that the graders are not required to read this material — please ensure that all important material fits under the page limit and includes pointers to the relevant parts of the appendix, where appropriate. You may refer to [1, 2] for examples of how to organise your space in the report and for how to visualise results effectively in limited space.

What will be graded Projects will be graded based on time and effort invested in the project. For this, you must show us evidence that you put in the work, e.g. with documentation of (a) results (negative or positive), and (b) roadblocks encountered and the steps taken to overcome them. Research projects are open-ended, so negative results (e.g. "the method I proposed does not outperform baselines") are to be expected. In fact, they are as valuable as positive results — so our evaluation will be the same for positive or negative results (with the grade replying on time and effort only).

0.3 LLM-use policy for projects

Please read the LLM policy carefully.

- LLMs may **not** be used to generate or substantially revise any written portion of the project deliverables, including the proposal, mid-review report and final report. Forbidden use-cases include generating an outline, drafting text, rewriting text (e.g. to avoid plagiarism detection), or paraphrasing text in your submission. All written content must be your own intellectual and stylistic work.
  - Grammar-only proofreading of text you wrote yourself is allowed.
- You can use the LLMs for other uses, such as programming/coding, helping understand background/literature, finding related works, conceptual clarifications, brainstorming ideas, or other means to understand the material.
- All LLM-provided information must be independently verified. In particular, avoiding incorrect or hallucinated information is entirely your responsibility. LLMs can frequently hallucinate math (equations/theorems), citations, references, experimental claims. For example, any citation that cannot be verified in DBLP, Google Scholar, Semantic Scholar or similar will be considered hallucinated.
- Any hallucinated or fabricated material included in your submission will be treated as academic misconduct and may result in significant penalties.
- You may be asked to explain and defend any part of your submission. "The LLM produced it" is not a valid defence for a lack of understanding or hallucinated content.
- **Disclosure**: Include an appendix describing: (a) whether you used an LLM, (b) for what purpose, (c) how you verified its outputs. Failure to disclose LLM usage may be treated as a violation of academic integrity.

Please also see the honour code for detailed rules.

1 Research Projects

The goal here is to conduct original research, which can be theoretical, numerical, applied, or any combination thereof. Given below are some seed ideas. You are welcome to work on your research project as long as it involves a component related to the topics covered in the course or to privacy in AI in general. If you are in doubt, talk to the instructor.

1.1 Improving and Auditing Copyright Guarantees for Language Models

Background Generative models of text (e.g. ChatGPT), images (e.g. stable diffusion), or speech are known to regurgitate parts of their training data. This is legally problematic when the regurgitated data is copyrighted (e.g. novels used in the training data of LLMs). Vyas et al. [3] defined *near-access freeness (NAF)*, a mathematical formalism of what it means to protect copyright. This formulation closely resembles differential privacy, as it is based on a certain KL divergence being small.

Consider generative models over outputs y given input x of the form $P(y|x)$, e.g. y is the output given by ChatGPT in response to your prompt x. Suppose we have access to a copyright-safe model $P_{\text{safe}}(\cdot|x)$ whose

training data is known to not contain any copyrighted text. The goal of NAF algorithms is to transform a risky model $P_{\text{risky}}(\cdot|x)$ into $P_\star(\cdot|x)$ such that $\text{KL}\big(P_\star(\cdot|x)\|P_{\text{safe}}(\cdot|x)\big) \leq K$ for all inputs x.

Just a few weeks ago, He et al. [4] gave such an algorithm, called anchored decoding, to generate text from an LLM in a token-by-token fashion. This algorithm enjoys a provable NAF guarantee at the sequence level. We'll cover this in April as a part of our advanced topics.

There are several open problems in this space, discussed below. Any one of them can be a suitable course project.

1.1.1 Improved Decoding Algorithms for k-NAF Protection

Summary:

- Topics covered: copyright, LLM-based text generation, designing algorithms for text generation with provable guarantees
- Additional background required: None
- Requirement of computational/GPU resources: significant. The free tier GPU of Colab/Kaggle would likely be insufficient for this project. However, free GPUs pooled over 3 people could be sufficient for proof-of-concept demonstrations.

The goal of this project is to develop new algorithms that can achieve a better copyright-utility tradeoff over anchored decoding. That is, they can generate higher quality (or more useful text) at the same level of provable NAF guarantees.

The current best algorithm for NAF relies on finding the closest distribution that satisfies the required KL guarantee. In the DP text generation literature, a similar approach was used to limit the Renyi divergence to give RDP bounds in [5]. However, the current state-of-the-art algorithm [6] for DP text generation uses aggregation in the space of logits (rather than in the space of probabilities). Then, one converts the logits into probabilities by applying the exponential mechanism. A good starting point for this project is to explore the application of similar ideas to copyright protection.

Potential Approach Here is a concrete proposal. Let ϕ_{risky} and ϕ_{safe} be the logits of the risk and safe LLMs respectively, such that the respective model probabilities $P_{\text{risky}} = \text{softmax}(\phi_{\text{risky}})$ and $P_{\text{safe}} = \text{softmax}(\phi_{\text{safe}})$ can be obtained by applying the softmax function over the logits. Consider the logit-space aggregation

$$\phi_\star = \alpha\, \phi_{\text{safe}} + \beta\, \text{clip}_C\big(\phi_{\text{risky}} - \phi_{\text{safe}}\big)\,,$$

where α, β, C are hyperparameters and $\text{clip}_C(v)$ makes sure that the each entry of the vector v is at most C. The goal is to find a NAF bound, i.e. bound the KL divergence between $P_\star = \text{softmax}(\phi_\star)$ and $Q = \text{softmax}(\phi_{\text{safe}})$, in terms of the constants α, β, C.

This can be achieved, for example, by using ideas from the exponential mechanism's zCDP guarantees[1] and post-processing. In particular, a zCDP guarantee of $R_\alpha(P\|Q) \leq \rho\alpha$ for all $\alpha > 1$ can be converted to a KL guarantee as $\text{KL}(P\|Q) = \lim_{\alpha\to 1^+} R_\alpha(P\|Q) \leq \rho$.

Pointers The code for anchored decoding is available online at `https://github.com/jacqueline-he/anchored-decoding`. One can directly use their implementation for reference.

[1] `https://differentialprivacy.org/exponential-mechanism-bounded-range/`

1.1.2 Auditing k-NAF Guarantees via Adversarial Attacks

Summary:

- Topics covered: copyright, LLM-based text generation, auditing
- Additional background required: None
- Requirement of computational/GPU resources: significant. The free tier GPU of Colab/Kaggle or free LLM API credits would likely be insufficient for this project, especially for problem 2. However, free GPUs pooled over 3 people could be sufficient for proof-of-concept demonstrations.

Suppose a developer claims that their model satisfies a certain NAF level and you believe that they are lying. The idea is to audit these guarantees, much like we might audit differential privacy guarantees (see e.g. [7] or `https://differentialprivacy.org/neurips23-op/`).

There are two issues: (1) testing for NAF guarantees for a given prefix x, and (2) finding a worst-case prefix x.

Problem 1: Testing NAF guarantees Suppose we are given a prefix x. Then, letting $p = P(\cdot|x)$ and $q = P_{\text{safe}}(\cdot|x)$, our goal is to ensure that the claimed NAF bound on $\text{KL}(p\|q)$ holds. Recall that $\text{KL}(p\|q) = \mathbb{E}_{Z\sim\text{PrivLoss}(p\|q)}[Z]$. Therefore, we can generate multiple i.i.d. samples $Z_1, \ldots, Z_n \sim \text{PrivLoss}(p\|q)$ and give a Monte Carlo estimate

$$\text{KL}(p\|q) \approx \frac{1}{n}\sum_{i=1}^{n} Z_i \,.$$

Unfortunately, this is not enough. If we want to claim that a NAF guarantee is false, we need to prove that $\text{KL}(p\|q) > K$ with high probability. Such bounds can be derived with concentration inequalities on the left tail:

$$\mathbb{P}\left(\text{KL}(p\|q) < \frac{1}{n}\sum_{i=1}^{n} Z_i - t\right) < \delta \,.$$

Generally, an empirical Bernstein inequality will give a tight concentration inequality here; see e.g. [8, Theorem 4].

Problem 2: Finding worst-case inputs The goal is to find a prompt x such that $\text{KL}\big(P(\cdot|x)\|P_{\text{safe}}(\cdot|x)\big)$ is as large as possible. This is also related to adversarial attacks.

In particular, the algorithm will:

1. propose a new set of prompts $x \in X$ by iteratively optimizing the previous one (this is also known as an evolutionary algorithm);
2. for each x, test the NAF guarantee.

The evolution step keeps half the best existing prompts and mutates them by making some structural changes, e.g. changing the system prompt, translating to a different language with a different script, etc. Note that elimination argument is analogous to the UCB algorithm for bandits (in particular, we use the LCB or lower concentration bound to eliminate candidates). On an unrelated note, similar evolutionary algorithms have also been used for differentially private synthetic data generation [9].

1.1.3 Measuring Empirical k-NAF Guarantees

An alternative to auditing is to empirically measure the level of NAF-copyright that is being violated by directly measuring the statistical divergences therein.

Measuring statistical divergences between complex distributions over text, images, or speech is possible: see the template given in [10, 11]. Your project can explore one of the following aspects of this topic:

- How to give a provable lower bound on the level of NAF-copyright violation using empirical measurements as per the above template?
- Does this measure of copyright satisfy intuitive properties we would expect? For example, does differential privacy help protect copyright as per your measurement? Also, if many repetitions of your data appear in the training set, does your measure detect more copyright leakage?

1.2 Signal Processing + DP: The Cepstral Mechanism for Learning with Correlated Noise

Summary:

- Topics covered: correlated noise mechanisms
- Additional background required: Knowledge of undergraduate-level signal processing will be necessary
- Requirement of computational/GPU resources: nominal. The free colab/Kaggle GPUs should be sufficient.

Correlated noise mechanisms provide an alterative to DP-SGD with Gaussian noise that is correlated across iterations; see the monograph [12]. Specifically, given i.i.d. Gaussian noise $z_t \sim \mathcal{N}(0, \sigma^2 I)$, correlated noise mechanisms perform the update

$$\theta_{t+1} = \theta_t - \eta(g_t + \hat{w}_t) \quad \text{where} \quad \hat{w}_t = c'_0 z_t + c'_1 z_{t-1} + \cdots + c'_t z_0 \,;$$

here, $\hat{w}_t$ is the correlated noise correlated by coefficients $c'_0, c'_1, \ldots$. Note that it reuses the old noise z_{t-1}, z_{t-2}, etc.

A key aspect in the design of correlated noise mechanisms is to find the noise correlations $c' = (c'_0, c'_1, \ldots)$. It is convenient to describe these coefficients by their generating functions:

$$f_{c'}(x) = \sum_{t=0}^{\infty} c'_t x^t \,.$$

Note that the noise coefficients define a filter acting on white Gaussian noise, and the generating function corresponds to the z-transform with $x = z^{-1}$. This project will apply ideas of cepstral approximations from signal processing to derive correlated noise mechanisms.

Background As we discuss in class and as described in [12, Section 2.9] (titled "An Approximation Theory Viewpoint"), the optimal choice of coefficients (in terms of the max loss) corresponds to the generating $f_{c'}(x) = \sqrt{1-x}$. For computational efficiency, we aim to use various approximations of this. For example:

- The banded and inverse banded mechanisms use polynomial approximations respectively to $1/f_{c'}(x)$ and $f_{c'}(x)$. This corresponds to approximating it with a FIR (finite impulse response) filter.
- The BLT mechanism [13] uses a rational approximation to $f_{c'}(x)$.

These approaches allow us to efficiently compute the correlated noise $\hat{w}_t$ while also getting good performance (this is described in detail in [12, Chap. 2]).

Project Idea This project will investigate cepstral approximations from signal processing in the context of correlated noise. A degree-d ceptstral approximation of $f_{c'}(x)$ takes a degree-d polynomial approximation $p(x) = \sum_{j=0}^{d} \ell_j x^j$ to $\log f_{c'}(x)$ to get the approximation:

$$f_{c'}(x) \approx f^{\text{cep}}(x) := \exp\left(\sum_{j=0}^{d} \ell_j x^j\right).$$

Typically, ℓ_j's are taken to the coefficients of the Taylor expansion of $\log f_{c'}(x)$.

A project can cover wide-ranging aspects from algorithm design to theoretical analysis to empirical experiments for machine learning. In particular:

- **Computational**: What is the space and time complexity of computing the correlated noise $\hat{w}_t$? **Hint.**[2]
- **Theory**: What is the utility (max loss) of this approximation? This will involve plugging in the cepstral approximation guarantee to [12, Theorem 2.26] (or for better constants, into [13, Prop. 4.1]).
- **Empirical**: How does the empirical max loss compare to the theoretical bounds (and with the other mechansims)? How good is it empirically for machine learning?

Even though some correlated noise mechanisms are suboptimal in terms of max loss, they can be quite useful in the machine learning context, e.g. [14].

1.3 Private Text Generation with Different DP Primitives

Background The state-of-the-art approach for private text generation [6] samples the next token is typically sampled using the exponential mechanism:

$$y_t \sim \text{softmax}(\bar{v}/\tau), \tag{1}$$

where $\bar{v}$ is the aggregated logit vector (across reference data) and τ is calibrated to satisfy a given zCDP level and logit clipping level.

This can be equivalently implemented as:

$$y_t = \arg\max_j \{\bar{v}_j + z_j\}, \tag{2}$$

where z_j are i.i.d. Gumbel random variables (with appropriate parameters). The equivalance between the two forms is known as the *Gumbel-max trick*.[3]

This alternate algorithm is known in the DP world as Report-Noisy-Max (RNM). Other noise distributions such as exponential/Laplace noise can also yield DP guarantees but induce different token selection behaviour. In particular, the exponential noise may concentrate more mass near the maximum logit compared to softmax sampling at comparable privacy levels, making it potentially more suitable for language models.[4]

[2] Hint: it can be done recursively by storing d previous i.i.d. noise vectors $z_{t-1}, \ldots, z_{t-d}$.

[3] See for details: `https://medium.com/swlh/on-the-gumbel-max-trick-5e340edd1e01`.

[4] Commonly used decoding algorithms for language model such as temperature rescaling and nucleus sampling focus on putting more mass on the head of the next-token distribution. RNM with exponential noise can potentially have a similar effect.

Project Idea What happens if we replace softmax-based sampling with Report-Noisy-Max rule with exponential noise? This change may result in one of two outcomes: it may either (a) improve utility by favouring high-logit tokens more strongly, or (b) degrade utility because LLMs are trained using cross-entropy loss, which is naturally aligned with softmax sampling. Finding this out will be an interesting observation.

Your project can involve (some aspects of) the following:

1. **Algorithm design**: Formalise RNM-based decoding under logit clipping and derive its differential privacy guarantees.
2. **Empirical evaluation**: Compare RNM and exponential-based decoding and evaluate downstream utility (see [6] for reference).
3. **Theoretical analysis**: If RNM decoding leads to utility improvements, compare the induced token distribution with softmax sampling, e.g. as a function of temperature and privacy level.
4. **Loss function design (if required)**: If RNM decoding reduces utility, you can try to design an alternative loss function to cross entropy that is aligned with the RNM-induced distribution. A natural option is maximum likelihood estimation(MLE) under the appropriate noise model. You will also have to design a numerically stable estimator that is linear in the vocabulary size, similar to cross entropy. **Hint**.[5] You can then verify if it recovers the utility for private decoding.

Starter code and pointers You can use the following repositories for reference and as the starter code:

- `https://github.com/cerai-iitm/InvisibleInk`
- `https://github.com/cerai-iitm/InvisibleInk-Experiments`

1.4 Cryptography + DP: Distributed Private Text Generation using MPC

Summary:

- Topics covered: LLM-based text generation with DP, multi-party computation (MPC).
- Additional background required: cryptography (MPC, secret sharing)
- Requirement of computational/GPU resources: moderate. It should be possible to have a proof-of-concept with free colab/Kaggle GPUs.

This project aims to extend the state-of-the-art method [6] for private text generation to the distributed setting with untrusted participants. The key is to use cryptographic tools for untrusted participants to work together, namely multi-party computation (MPC). The combination of DP with MPC tools has been extensively studied in other contexts, e.g. [15, 16, 17].

[5] You can estimate the probability with numerical quadrature in a way that is compatible with automatic differentiation.

Setting Suppose there are n parties where party i holds a private document r_i. At time t, all parties have a common prefix $y_{<t}$ and wish to sample the next token y_t.

Each party computes

$$v_i = \phi(\cdot \mid r_i, y_{<t}),$$

where ϕ is a shared LLM logit function. The aggregate logit is (ignoring specific details about careful clipping)

$$\bar{v} = \frac{1}{n} \sum_{i=1}^{n} v_i.$$

In a trusted setting, a central aggregator would compute $\bar{v}$ and apply the private decoding rule Here, parties are honest-but-curious and do not trust a central aggregator.

Project Idea Design an MPC protocol that computes Eq. (2), i.e.

$$y_t = \arg\max_j \{\bar{v}_j + z_j\}$$

without revealing the individual logits v_i, any intermediate aggregates, or individual noise shares.

Some additional algorithmic components might be needed:

MPC typically requires us to split the noise z_j into shares $z_{i,j}$ held by user i such that $z_j = \sum_i z_{i,j}$. That is, user i generated $z_{i,j}$ independently of others (e.g. on their own hardware) such that their sum $\sum_i z_{i,j}$ has the correct distribution. Unfortunately, the Gumbel noise z_j implied by the exponential mechanism in (2) is not *infinitely divisible*,[6] so we can never find i.i.d. shares of Gumbel noise. So we'll have to replace the Gumbel noise by some other infitely divisible noise distributions, e.g. the exponential distribution from Section 1.3.

Your project can then involve the following components:

- **MPC Protocol**: Design an MPC protocol to compute

  $$\arg\max_j \left\{ \frac{1}{n} \sum_{i=1}^{n} (v_{i,j} + z_{i,j}) \right\}$$

  under the semi-honest model. Functional secrete sharing is a compelling option, but feel free to use other tools such as secure comparison protocols. Analyse communication and computation complexity as a function of vocabulary size and number of parties.

- **Full algorithm and guarantees**: Give the resulting end-to-end algorithm and prove differential privacy and security under the honest-but-curious model.

- **Empirical evaluation**: Implement a prototype and measure the runtime per decoding step, communication cost, scalability with n, and utility relative to the centralised private baseline.

Starter code and pointers Same as for Section 1.3.

[6] See [18] for background on infinite divisibility of distributions.

1.5 Privacy Attacks on Kernel Machines

Summary:

- Topics covered: Membership inference attacks, Kernel methods
- Additional background required: kernel machines (e.g. kernel ridge regression, kernel SVM)
- Requirement of computational/GPU resources: moderate.

Kernel machines such as kernel ridge regression (KRR) and kernel SVM are classical non-parametric learning methods. Given training data $\{(x_i, y_i)\}_{i=1}^n$ with $x_i \in \mathbb{R}^d$, a kernel method is based on a positive definite kernel $k(x, x') = \langle \phi(x), \phi(x') \rangle$ without explicitly constructing the (possibly infinite dimensional) feature map ϕ. Common examples include the Gaussian kernel $k(x, x') = \exp\left(\frac{1}{2b^2}\|x - x'\|_2^2\right)$ and the ℓ_2-Laplace kernel $k(x, x') = \exp\left(\frac{1}{2b}\|x - x'\|_2\right)$. More structured kernels are possible for specific domains such as images and/or text, including those based on deep nets.

A kernel machine makes predictions at a query point x by evaluating

$$f(x) = \sum_{i=1}^{n} \alpha_i k(x, x_i),$$

where the x_i's are the training data points, and the weights $\alpha_1, \ldots, \alpha_n$ represent the model. In particular, the learned model is a weighted combination of kernel evaluations against training points. This explicit dependence on the training data may introduce privacy risks — the goal of this project is to study the extent of these risks. In particular, prior work has demonstrated that kernel machines are quite susceptible to data reconstruction attacks [19].

Project Idea You will develop a membership inference attack for kernel machines.

Specifically, assume an adversary has black-box access to the trained model f. Given a candidate point $(x^\star, y^\star)$, the adversary's goal is ascertain whether this point was part of the training set. The attacker may query f at $x^\star$ and other adaptively chosen points. The attacker is also assumed to know the kernel, learning algorithm and all hyperparameters (bandwidth, regularisation parameter, etc.).

Your project can involve the following steps:

- Theoretical analysis (for kernel ridge regression): You can derive theoretical bounds on how susceptible a data point is to membership inference for kernel ridge regression. In particular, the effect of a data point is measured by the diagonal entries of the so-called hat matrix $H = (K + \lambda I)^{-1}K$, where K is the kernel matrix.[7] It would be even more interesting to analytically describe the dependence of problem parameters (e.g. regularisation parameter, eigen-decay of the kernel, etc.) on the success of the membership inference attack.[8]
- Design a membership inference attack. That is, find a test statistic that can distinguish between members and non-members. Try to develop test statistics that are tailored to the problem (e.g. based on the hat matrix) and compare with baselines (e.g. loss of the datapoint).

[7] In particular, the leave-one-out effect of a data point of leaving out a data point i depends on the diagonal entry H_{ii}.

[8] The instructor can provide more detailed references/reading on the theoretical analysis upon request.

- Design and insert canary data points that are easy to detect with membership inference. You can use knowledge of the data distribution for this task (e.g. via held-out examples). **Hint**.[9]

Evaluate the membership inference attack using TPR@FPR$\approx O(1/n)$.

Starter code and pointers You can use this starter code: `https://gist.github.com/krishnap25/a723af46b73ca68d9b587db042c3ca7b`. This starter code is due to Prof. Parthe Pandit from IIT Bombay, who was kind enough to create it specifically for our course projects.

The Laplace kernel is a good default for experiments, as it is relatively insensitive to the bandwidth hyperparameter b (where the Gaussian kernel is highly sensitive to tuning the bandwidth). For regression, it also makes sense to first consider the so-called ridgeless regression, i.e. ridge regression in the limit that $\lambda \to 0^+$.

1.6 The Effect of Distribution Shifts on Pretraining for Private Learning

Summary:

- Topics covered: empirical measurements of distribution shifts. This is a purely empirical project (there is no theory involved).
- Additional background required: None
- Requirement of computational/GPU resources: significant.

Background It is well-understood that pretraining a model (on public data) is often essential to get good performance on private data [21, 22]. However, the extent to which the private data and public data are similar to each other is a key determining factor.

Project Problem Design a synthetic benchmark where you can control the extent to which a pretraining dataset and finetuning dataset align with each other. Some examples can include:

- **Images**: You could consider pretraining on various subsets of the SVHN dataset [23] and fine-tune (with differential privacy) on MNIST. You can measure the gap between the pretraining and finetuning dataset with the approach presented in [11], for instance. You can add a layer of complexity by using SVHN for unsupervised pretraining (e.g., using an auto-encoder) rather than supervised pretraining for classification.
- **Language**: You could design distributions over tokens using a Markov chain (of order 3, 4, or 5, with variable complexity) for a language modeling task. The pretraining and finetuning data distributions could correspond to Markov chains with transition probabilities that are slightly different. You can add a layer of complexity by drawing the pretraining and finetuning distributions from transformer language models with parameters that are slightly different. You can measure the gap between the pretraining and finetuning dataset with the approach presented in [10, 11]. For a more realistic dataset, see [24].

Then, use this benchmark to empirically quantify the effect of the distribution shift between the pretraining data and the finetuning data. This project is primarily empirical but it might be possible to manage on free GPU resources (although more GPUs will certainly help).

[9] Canaries are usually designed to be out-of-distribution. See, for example, [20].

2 Implementation + Open-Sourcing or Benchmarking

These projects are primarily oriented at implementing algorithms and running them on popular datasets. The exact emphasis of the project can be on:

- **Benchmarking**: Implement an existing algorithm and perform a thorough empirical comparison to baselines. The main goal here is to perform the detailed empirical comparison, while producing useful software is a lower priority.
- **Open-source packages**: Contribute an implementation of an existing algorithm to existing popular packages or create your own. While it is important to benchmark the algorithms on common datasets, there is a greater emphasis on good software engineering practices to release software that is useful to many other people. A successful project will result in an open-source package that is publicly available on GitHub and possibly on PyPI.

You can choose which combination of the two components you want for your project.

2.1 Comparison of Large-Scale DP Training Frameworks

Several software packages exist to implement DP fine-tuning of LLMs, but there is no single accepted standard. For instance, PyTorch and HuggingFace Transformers provide a universal default for non-private deep learning and language modeling — this makes it easy to get started. However, when it comes to DP training, there are various pros and cons to consider and there is no clear winner. So a newcomer to the field does not have any clear information on where to get started.

This project will fix this issue by conducting a detailed experimental comparison of various packages to make a concrete recommendation for future users.

Detailed Notes

- Packages to compare include Opacus, DP-transformers, Fast-DP, and Jax Privacy.
- The following factors are worth comparing:
  - compatibility with HuggingFace transformers out-of-the-box (i.e. without the need to define custom layers)
  - compatibility with parallel/distributed training tools and methods such as multi-GPU training
  - per-iteration training time overhead (as we know, DP training is time intensive)
  - ease of integrating the HuggingFace Trainers and other abstractions that make HuggingFace packages so easy to use.
- Also, it is worth considering how easy it is to implement new DP training algorithms (e.g. with error feedback, see Section 2.3).

Other deliverables In addition to course project deliverables, it is worthwhile to produce a short blog post with these comparisons, and an accompanying jupyter notebook for people to use directly. Given the original motivation behind this project, such a blog post will be highly impactful and influential in the community.

2.2 Benchmarking of Correlated Noise Mechanisms for ML Problems

Correlated noise mechanisms for private learning have been quite influential in theory and practice. The latest in the field is applications to machine learning problems, where we run over multiple epochs (as opposed to the streaming setting).

The project will benchmark and empirically compare the performance a few such mechanisms:

- BLT mechanism: proposed in [13] and extended to the multi-epoch setting of machine learning setting in [25].
- Banded Toeplitz mechanism: as discussed in class, where the C matrix is banded [26, 27].
- Inverse banded mechanism: a variant of the above where the C^{-1} matrix is taken to be banded [28].

Background The foundations are described in Chapter 2 of the correlated noise monograph [12] and extensions to the machine learning setting are described in Chapter 3. Of course, the parameters of these mechanisms have to be optimized — the code to do that is provided in the Jax-Privacy codebase at `https://github.com/google-deepmind/jax_privacy/tree/main/jax_privacy/matrix_factorization`.

Project Goal Compare the above mechanisms empirically in the following settings:

- in terms of the factorization losses such as the max loss and RMS loss;
- in terms of the empirical performance on model training, e.g. with MNIST or CIFAR.

The first goal should be to complete the above two tasks without amplification by sampling. Concretely, this will require writing efficient code for DP-SGD with correlated noise, compatible with each of the above mechanisms. The code to actually find the C or C^{-1} matrices is available from the jax-privacy repo linked above (for the inverse banded mechanism, the code can be found in the paper [28].

Sample training code can be found at `https://github.com/npkalinin/Matrix-Factorization-DP-Tra` `tree/main` Note: this repo is research code and has not been optimized or cleaned. The output of the project is expected to adhere to higher standards of software development.

Stretch Goals The next steps can be to repeat the above two tasks but with amplification using balls-in-bins sampling (a.k.a. random allocation) — this is an alternative sampling scheme that is more compatible with correlated noise and is closer to practical implementations. The code to derive the amplified privacy guarantees can be found online.[10] Alternatively, you can empirically evaluate the three (unamplified) mechanisms for other tasks, such as language model finetuning.

2.3 Differentially Private Learning with Error Feedback

The gradient clipping step of DP-SGD introduces a bias relative to the original problem we wished to solve. Zhang et al. [29] proposed an algorithm called Dice-SGD that avoids this clipping bias using a technique known as error feedback. However, no public implementation of this algorithm exists, and it is unclear what the root cause of its empirical improvements is.

[10] `https://github.com/google-deepmind/jax_privacy/blob/main/examples/balls_in_bins_accounting.py`

Implementation Implement it in the Opacus library and extensively test and verify your implementation. (Testing will invariably involve some amount of benchmarking.) After your project is complete, we can try to work with the creators of Opacus to open-source your implementation if you are interested (note that this might require extra work from your end after the end of the course).

Benchmarking Compare its performance to DP-SGD across a range of clipping parameters. Check the following:

- How sensitive is Dice-SGD to its clip norm hyperparameters? That is, how robust is each algorithm to misspecification of the clipping hyperparameters?
- Does Dice-SGD outperforms DP-SGD at (a) the best hyperparameters for each, and (b) heuristically chosen (but possibly suboptimal) hyperparameter choices?

You can compare the algorithms on MNIST and/or CIFAR datasets.

2.4 Benchmark DP-SGD on Chest X-ray Data

Train a differentially private image classification model on Chest X-ray data from (a) Kaggle[11], and/or (b) the IN-CXR dataset, an Indian dataset released by ICMR[12].

Benchmark the effect of using different amounts of the original data (noting that it leads potentially to a class imbalance), and various hyperparameters for DP-SGD. You can also study and implement the various tricks employed in [30].

2.5 Unlearning with Differential Privacy

Unlearning some training data is a crucial problem, with lawmakers in various countries debating a "right to be forgotten". A recent approach called *Langevin Unlearning* [31] uses ideas related to differential privacy. The paper only gives experiments on simple binary logistic regression tasks (using two classes of MNIST and CIFAR). It would be interesting to benchmark this approach along with other baselines on training MNIST/CIFAR with neural nets. (Note that their guarantees are likely to be very loose in that setting, leading to poor performance. But it is important to know where we stand on this.)

3 In-Depth Paper Analysis

Replicate the results of one theoretical paper or a subset of results from a line of theoretical papers. Here are some suggestions:

Papers that are highly interesting:

- Theoretical analysis of hallucinations in language models [32]. This is not directly related to privacy but has strong privacy implications.
- Privacy filters: Privacy filters turn static data protection into a dynamic "game" by allowing researchers to adaptively query datasets until privacy budget of a specific data point is extinguished. They are quite useful for modern AI, ensuring that iterative systems like RAG systems do not accidentally bankrupt

[11] https://www.kaggle.com/datasets/tawsifurrahman/tuberculosis-tb-chest-xray-dataset
[12] https://nirt.res.in/html/xray.html

an individual's privacy budget. A nice course project will combine a foundational paper and a recent paper:

  - Foundational papers: [33, 34] – pick one
  - Recent papers on their limits: [35, 36] – pick one

- Privacy amplification by iteration: In strongly convex optimization, subsequent iterations can better protect the privacy of data seen earlier in training, a phenomenon known as privacy amplification by iteration [37]. This is a classic paper with very interesting proof techniques.

- The convergence of the privacy loss in strongly convex optimization [38]. Again, this paper uses very interesting proof techniques based on optimal transport (see e.g. [39] for a gentle introduction) — this is a very mathematically rich area in the intersection of mathematics, probability/statistics, and optimization.

- DP-inspired techniques for credit attribution for ML models [40]. That is, if a model makes a certain prediction, which data points are responsible for it?

- Quantum differential privacy [41]: how to formalize differential privacy for quantum algorithms (i.e. algorithms that can leverage quantum computers).

  - The goal is to explain the material to a machine learning audience with minimal quantum background.
  - Need some prior background in quantum algorithms and/or quantum info theory.
  - For a more gentle introduction, see [42].

- DP protections for privacy attacks beyond worst-case membership inference [43].

- Correlated noise mechanisms to solve problems in theoretical computer science problems [44].

- Implicit privacy in low-rank adaptation (LoRA) for parameter-efficient fine-tuning [45].

Other suggestions of interesting papers:

- Theoretical analysis of data reconstruction attacks for linear regression [46].

- Implicit DP guarantees in KL-regularized bandits and RLHF [47]. The interesting immediate question is how to do privacy accounting here.

- User-level metric differential privacy [48] (i.e. we have greater privacy guarantees for inputs that are more similar to each other). This is suitable for folks with broad knowledge of probability.

- Efficient and near-optimal streaming differential privacy with correlated noise mechanisms [13]. This uses tools from approximation theory: it is a good choice for folks with a background in theoretical computer science.

- Privacy accounting for correlated noise mechanisms including amplification by sampling [49]. This is a very interesting paper that uses accounting based on dominating distributions on the privacy losses directly.

- Fingerprinting lower bounds: an interesting technique to derive lower bounds on the performance of private algorithms (i.e. bounds of the form "any (ε, δ)-DP mechnism for this particular task should have an error of at least ...". Papers: [50] or [51] (recommended as it is somewhat easier to parse).

References

[1] Christopher A Choquette-Choo, Krishnamurthy Dvijotham, Krishna Pillutla, Arun Ganesh, Thomas Steinke, and Abhradeep Thakurta. Correlated Noise Provably Beats Independent Noise for Differentially Private Learning. In *International Conference on Learning Representations*, 2024.

[2] Krishna Pillutla, Galen Andrew, Peter Kairouz, H Brendan McMahan, Alina Oprea, and Sewoong Oh. Unleashing the Power of Randomization in Auditing Differentially Private ML. *Advances in Neural Information Processing Systems*, 36, 2023.

[3] Nikhil Vyas, Sham M Kakade, and Boaz Barak. On Provable Copyright Protection for Generative Models. In *International Conference on Machine Learning*, pages 35277–35299. PMLR, 2023.

[4] Jacqueline He, Jonathan Hayase, Wen-tau Yih, Sewoong Oh, Luke Zettlemoyer, and Pang Wei Koh. Anchored Decoding: Provably Reducing Copyright Risk for Any Language Model. *arXiv preprint arXiv:2602.07120*, 2026.

[5] James Flemings, Meisam Razaviyayn, and Murali Annavaram. Differentially private next-token prediction of large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 4390–4404, 2024.

[6] Vishnu Vinod, Krishna Pillutla, and Abhradeep Thakurta. InvisibleInk: High-Utility and Low-Cost Text Generation with Differential Privacy. In *NeurIPS*, 2025.

[7] Ivars Namatevs, Kaspars Sudars, Arturs Nikulins, and Kaspars Ozols. Privacy auditing in differential private machine learning: The current trends. *Applied Sciences*, 15(2):647, 2025.

[8] Andreas Maurer and Massimiliano Pontil. Empirical bernstein bounds and sample variance penalization. *arXiv preprint arXiv:0907.3740*, 2009.

[9] Chulin Xie, Zinan Lin, Arturs Backurs, Sivakanth Gopi, Da Yu, Huseyin A Inan, Harsha Nori, Haotian Jiang, Huishuai Zhang, Yin Tat Lee, et al. Differentially private synthetic data via foundation model apis 2: Text. In *ICML*, 2024.

[10] Krishna Pillutla, Swabha Swayamdipta, Rowan Zellers, John Thickstun, Sean Welleck, Yejin Choi, and Zaid Harchaoui. MAUVE : Measuring the Gap Between Neural Text and Human Text using Divergence Frontiers. *Advances in Neural Information Processing Systems*, 34:4816–4828, 2021.

[11] Krishna Pillutla, Lang Liu, John Thickstun, Sean Welleck, Swabha Swayamdipta, Rowan Zellers, Sewoong Oh, Yejin Choi, and Zaid Harchaoui. Mauve Scores for Generative Models: Theory and Practice. *Journal of Machine Learning Research*, 24(356):1–92, 2023.

[12] Krishna Pillutla, Jalaj Upadhyay, et al. Correlated noise mechanisms for differentially private learning. 2025.

[13] Dvijotham Krishnamurthy, H Brendan McMahan, Krishna Pillutla, Thomas Steinke, and Abhradeep Thakurta. Efficient and Near-optimal Noise Generation for Streaming Differential Privacy. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, 2024.

[14] Nikita P. Kalinin, Ryan McKenna, Rasmus Pagh, and Christoph H. Lampert. DP-λCGD: Efficient Noise Correlation for Differentially Private Model Training. *ArXiv*, 2026.

[15] Cynthia Dwork, Krishnaram Kenthapadi, Frank McSherry, Ilya Mironov, and Moni Naor. Our data, ourselves: Privacy via distributed noise generation. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 486–503. Springer, 2006.

[16] Peter Kairouz, Ziyu Liu, and Thomas Steinke. The distributed discrete gaussian mechanism for federated learning with secure aggregation. In *International conference on machine learning*, pages 5201–5212, 2021.

[17] Rasmus Pagh and Nina Mesing Stausholm. Infinitely divisible noise in the low privacy regime. In *International Conference on Algorithmic Learning Theory*, pages 881–909. PMLR, 2022.

[18] Arup Bose, Anirban Dasgupta, and Herman Rubin. A contemporary review and bibliography of infinitely divisible distributions and processes. *Sankhyā: The Indian Journal of Statistics, Series A*, pages 763–819, 2002.

[19] Daniel Barzilai, Yuval Margalit, Eitan Gronich, Gilad Yehudai, Meirav Galun, and Ronen Basri. Querying Kernel Methods Suffices for Reconstructing their Training Data. *arXiv preprint arXiv:2505.19019*, 2025.

[20] Matthew Jagielski, Jonathan Ullman, and Alina Oprea. Auditing differentially private machine learning: How private is private sgd? *Advances in Neural Information Processing Systems*, 33:22205–22216, 2020.

[21] Xuechen Li, Florian Tramer, Percy Liang, and Tatsunori Hashimoto. Large language models can be strong differentially private learners. 2022.

[22] Arun Ganesh, Mahdi Haghifam, Milad Nasr, Sewoong Oh, Thomas Steinke, Om Thakkar, Abhradeep Guha Thakurta, and Lun Wang. Why is public pretraining necessary for private model training? In *International Conference on Machine Learning*, pages 10611–10627. PMLR, 2023.

[23] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Baolin Wu, Andrew Y Ng, et al. Reading digits in natural images with unsupervised feature learning. In *NeurIPS workshop on deep learning and unsupervised feature learning*, volume 2011, page 4, 2011.

[24] Mansi Sakarvadia, Aswathy Ajith, Arham Khan, Nathaniel Hudson, Caleb Geniesse, Kyle Chard, Yaoqing Yang, Ian Foster, and Michael W Mahoney. Mitigating memorization in language models. In *ICLR*, 2025.

[25] H Brendan McMahan, Zheng Xu, and Yanxiang Zhang. A hassle-free algorithm for private learning in practice: Don't use tree aggregation, use blts. *arXiv preprint arXiv:2408.08868*, 2024.

[26] Nikita Kalinin and Christoph H Lampert. Banded Square Root Matrix Factorization for Differentially Private Model Training. In *NeurIPS*, volume 37, pages 17602–17655, 2024.

[27] Ryan McKenna. Scaling up the Banded Matrix Factorization Mechanism for Differentially Private ML. In *ICLR*, 2025.

[28] Nikita P Kalinin, Ryan McKenna, Jalaj Upadhyay, and Christoph H Lampert. Back to Square Roots: An Optimal Bound on the Matrix Factorization Error for Multi-Epoch Differentially Private SGD. In *ICLR*, 2026.

[29] Xinwei Zhang, Zhiqi Bu, Zhiwei Steven Wu, and Mingyi Hong. Differentially Private SGD Without Clipping Bias: An Error-Feedback Approach. In *International Conference on Learning Representations*, 2024.

[30] Soham De, Leonard Berrada, Jamie Hayes, Samuel L Smith, and Borja Balle. Unlocking high-accuracy differentially private image classification through scale. *arXiv preprint arXiv:2204.13650*, 2022.

[31] Eli Chien, Haoyu Wang, Ziang Chen, and Pan Li. Langevin Unlearning: A New Perspective of Noisy Gradient Descent for Machine Unlearning. *arXiv preprint arXiv:2401.10371*, 2024.

[32] Adam Tauman Kalai and Santosh S Vempala. Calibrated language models must hallucinate. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, pages 160–171, 2024.

[33] Ryan M Rogers, Aaron Roth, Jonathan Ullman, and Salil Vadhan. Privacy odometers and filters: Pay-as-you-go composition. *Advances in Neural Information Processing Systems*, 29, 2016.

[34] Vitaly Feldman and Tijana Zrnic. Individual privacy accounting via a renyi filter. *Advances in Neural Information Processing Systems*, 34:28080–28091, 2021.

[35] Matthew Regehr, Bingshan Hu, Ethan Leeman, Pasin Manurangsi, Pierre Tholoniat, and Mathias Lécuyer. Natural privacy filters are not always free: A characterization of free natural filters. *arXiv preprint arXiv:2602.15815*, 2026.

[36] Long Tran, Antti Koskela, Ossi Räisä, and Antti Honkela. f-differential privacy filters: Validity and approximate solutions. *arXiv preprint arXiv:2602.06756*, 2026.

[37] Vitaly Feldman, Ilya Mironov, Kunal Talwar, and Abhradeep Thakurta. Privacy amplification by iteration. In *2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 521–532. IEEE, 2018.

[38] Jason Altschuler and Kunal Talwar. Privacy of noisy stochastic gradient descent: More iterations without more privacy loss. *Advances in Neural Information Processing Systems*, 35:3788–3800, 2022.

[39] Gabriel Peyré and Marco Cuturi. *Computational optimal transport: With applications to data science*. Now Foundations and Trends, 2019.

[40] Roi Livni, Shay Moran, Kobbi Nissim, and Chirag Pabbaraju. Credit attribution and stable compression. *Advances in Neural Information Processing Systems*, 37:2663–2685, 2024.

[41] Christoph Hirche, Cambyse Rouzé, and Daniel Stilck França. Quantum differential privacy: An information theory perspective. *IEEE Transactions on Information Theory*, 69(9):5771–5787, 2023.

[42] William M Watkins, Samuel Yen-Chi Chen, and Shinjae Yoo. Quantum machine learning with differential privacy. *Scientific Reports*, 13(1):2453, 2023.

[43] Marika Swanberg, Meenatchi Sundaram Muthu Selva Annamalai, Jamie Hayes, Borja Balle, and Adam Smith. Beyond the worst case: Extending differential privacy guarantees to realistic adversaries. *arXiv preprint arXiv:2507.08158*, 2025.

[44] Joel Daniel Andersson, Palak Jain, and Satchit Sivakumar. Improved accuracy for private continual cardinality estimation in fully dynamic streams via matrix factorization. *arXiv preprint arXiv:2601.02257*, 2026.

[45] Yaxi Hu, Johanna Düngler, Bernhard Schölkopf, and Amartya Sanyal. LoRA and Privacy: When Random Projections Help (and When They Don't). *arXiv preprint arXiv:2601.21719*, 2026.

[46] Leonardo Iurada, Simone Bombari, Tatiana Tommasi, and Marco Mondelli. A law of data reconstruction for random features (and beyond). *arXiv preprint arXiv:2509.22214*, 2025.

[47] Yizhou Zhang, Kishan Panaganti, Laixi Shi, Juba Ziani, and Adam Wierman. KL-regularization Itself is Differentially Private in Bandits and RLHF. *arXiv preprint arXiv:2505.18407*, 2025.

[48] Jacob Imola, Amrita Roy Chowdhury, and Kamalika Chaudhuri. Metric Differential Privacy at the User-Level. *arXiv preprint arXiv:2405.02665*, 2024.

[49] Christopher A Choquette-Choo, Arun Ganesh, Thomas Steinke, and Abhradeep Thakurta. Privacy amplification for matrix mechanisms. In *International Conference on Learning Representations*, 2023.

[50] Mark Bun, Jonathan Ullman, and Salil Vadhan. Fingerprinting Codes and the Price of Approximate Differential Privacy. In *Proceedings of the forty-sixth annual ACM symposium on Theory of computing*, pages 1–10, 2014.

[51] Thomas Steinke and Jonathan Ullman. Between pure and approximate differential privacy. *arXiv preprint arXiv:1501.06095*, 2015.